

Why GL-iNet — segmentation, Tailscale overlay, multi-WAN, and beyond this event

This design already uses GL-iNet Slate AX (GL-AXT1800) routers throughout (previously A-1300s, replaced 2026-10-06 — see the comparison below; also `docs/topology.md`, `docs/open-questions.md`). This document is the fuller "why" — the architectural reasoning behind putting a small router in every theatre instead of a flatter network, why Tailscale is the routing layer between them rather than site-to-site VLANs alone, the WAN-flexibility features this build doesn't currently use but is worth knowing about, and — since this is genuinely reusable hardware, not a one-event purchase — where else this same GL-iNet + Tailscale pattern earns its keep beyond a 12-theatre live event.

Network segmentation: one subnet per theatre

Each theatre is its own isolated /24 (192.168.2.0/24 through 192.168.13.0/24 — see `docs/ip-address-map.md`), NAT'd behind its own GL-iNet router, rather than 12 theatres' worth of devices sharing one flat network. Three concrete benefits fall out of that, beyond the obvious "keeps the IP plan tidy":

Individual DHCP servers. Each router runs its own DHCP scope for its own theatre. A DHCP problem — exhaustion, a rogue second DHCP server accidentally plugged in, a lease conflict — is contained to that one theatre's ~8 devices. On a flat network with one central DHCP server, the same fault takes out DHCP for the entire event at once. With 12 theatres + 2 VMix nodes + the mothership as 15 separate broadcast domains on the theatre-facing network (the edit suite's own 10GbE LAN is a further one — see `docs/live-editing.md`), there are 15 separate DHCP fault domains instead of one.

Multicast/broadcast containment. mDNS, SSDP, and NDI's own local discovery are all broadcast/multicast-based, which only works within a single L2 broadcast domain. Flatten 12 theatres onto one network and multicast traffic doesn't just fail to reach where it's needed — it also *floods everywhere it isn't*, since every discovery packet from every theatre reaches every other theatre's devices too. This design already runs into exactly that limitation on purpose: NDI's backup path needed a **unicast** NDI Discovery Server (see `docs/streaming-flow.md`) precisely because Tailscale's point-to-point routing (like any routed, non-bridged network) doesn't carry multicast across the theatre boundary. Segmentation isn't just tolerating that limitation — it's the reason cross-theatre multicast noise was never a problem to begin with, for anything that doesn't specifically need to be reachable across theatres.

Security — blast-radius containment. A compromised or simply misbehaving device in one theatre (malware on a presenter's laptop, a flaky ATEM firmware bug, an ARP-spoofing or DHCP-starvation attempt) is contained to that theatre's subnet. It cannot see, scan, or reach another theatre's devices or traffic, because there's no route between them — `config/tailscale-acl.json` only permits `theatre → mothership` and `mothership → theatre`, never `theatre → theatre`. This is the same point already made elsewhere in this repo — "VLAN grouping = physical uplink cabling only, cross-theatre isolation is enforced by Tailscale ACLs, not by the VLANs themselves" — worth restating here as the *reason* per-theatre segmentation is worth doing at all: it's not really about the subnets, it's about giving the ACL layer clean boundaries to enforce isolation across.

Tailscale as the overlay routing layer

Segmentation alone doesn't connect the theatres to the mothership — something has to route between the isolated subnets on purpose, while still keeping them isolated from each other. That's Tailscale's job here (full detail in `docs/tailscale.md`), and it's worth being explicit about why an overlay mesh is the right tool for that instead of relying on the venue's own physical network (VLAN trunking, static routes, a traditional site-to-site VPN concentrator):

- **Direct point-to-point tunnels, not routed via one central concentrator.** Every node-to-node WireGuard connection is negotiated independently and prefers a direct path when one exists — no single choke point every packet has to

pass through, which matters for something latency-sensitive like SRT.

- **Doesn't depend on the underlying network cooperating.** A traditional VLAN-routed design needs the venue's switches trunking the right VLANs to the right ports, correctly, everywhere. Tailscale's NAT traversal (direct when possible, DERP relay as fallback — see `docs/tailscale.md`) works regardless of what the underlying physical topology actually looks like, which matters a lot when the venue's network isn't something this design controls in the first place (see `docs/bandwidth-analysis.md` — the venue-supplied-VLAN context that document was written against).
- **Isolation enforced at the overlay, not hoped-for at the physical layer.** The ACLs in `config/tailscale-acl.json` are what actually stop theatre-to-theatre traffic — independent of whether the venue's VLANs are configured exactly as planned. If a VLAN turns out flatter than expected, or a switch port is misconfigured, cross-theatre isolation still holds, because it was never depending on the physical segmentation to begin with.
- **No manual routing tables.** Each router advertises its own subnet (`--advertise-routes`) and accepts the others' where ACLs allow (`--accept-routes`) — see `config/tailscale-up-all-devices.sh`. Nobody is hand-maintaining a routing table across 14+ devices.
- **Identity independent of location.** A router's Tailscale identity is its WireGuard key, not an IP address on a specific network. That matters directly for the multi-WAN/failover discussion below: a router can switch from its wired uplink to WiFi or cellular mid-event, and every other node on the tailnet keeps reaching it under the same identity, with no reconfiguration anywhere else.

Multi-WAN: wired primary, WiFi/cellular failover

This build's current default is a single hardwired WAN per theatre (see `docs/topology.md`) — but GL-iNet's broader lineup supports genuine multi-WAN failover: multiple uplink sources (wired Ethernet, WiFi acting as a WAN client, a USB cellular modem) with automatic failover between them, no manual cable-swap or on-site intervention required. This is a firmware 4.x feature (**NETWORK → Multi-WAN**: Failover or Load Balance across Ethernet, Repeater, Tethering and Cellular) that even the small A-1300's own user guide documents, not just the bigger models — what differs per model is which WAN *media* it has to fail over to (neither the A-1300 nor the Slate AX has a built-in modem, so cellular means a USB dongle or a tethered phone) (GL.iNet Multi-WAN guide, A-1300 user guide). For a deployment where most theatres run unattended for long stretches, that matters — a lost wired connection mid-session isn't something someone can necessarily walk over and fix before it costs real time.

Two failover directions are both worth having, and they're mirror images of each other:

- **Wired as primary, WiFi/cellular as backup** — this build's actual recommendation. If the venue's wired drop to a theatre fails (a bad cable, a switch fault, an ISP issue on the venue's side), the router falls back to venue WiFi or a cellular USB modem automatically. Bandwidth degrades under failover — that's expected and fine, it's a "keep the tailnet reachable and keep functioning at reduced quality" fallback, not a silent full-quality substitute.
- **WiFi as primary, for events that don't need this build's full streaming load.** Not every event this hardware gets reused for will run 12 theatres' worth of continuous SRT/NDI simulcast and near-real-time ISO ingest. For a lighter event — say, a conference room doing PowerPoint sync and basic control traffic, no live camera program feed — venue WiFi alone may comfortably cover it, and using each theatre's GL-iNet as a WiFi client eliminates the need for a wired drop to every room at all. Faster to set up, and avoids paying for (or waiting on) cabling runs the event genuinely doesn't need at that load level. This is the same "match infrastructure to actual load" reasoning `docs/bandwidth-analysis.md` already applies to VLAN count, just applied to WAN medium instead.

Limiting devices on pay-per-device venue WiFi

Some venues charge for WiFi **per connected device**, not by bandwidth tier. Connect every individual device in a theatre — the ATEM, BirdDog Play, both PowerPoint laptops, both VT laptops, the control laptop — directly to venue WiFi, and that's up to 7 billable devices per theatre. Put one GL-iNet router in each theatre instead, and the venue sees **one** device — the router — while everything behind it reaches the internet NAT'd through that single connection.

Worked example — real numbers from two major London exhibition venues' published 2025/2026 rate cards (cited as market examples of the pay-per-device model, not as this design's venue):

	ExCeL London	Olympia London (via eForce)
Single-device WiFi (per device)	£128 (advance) – £184 (late)	£171 (standard) – £205 (late)
Smallest bulk tier	10 devices: £647–932	6 devices: £519–623
7 devices, billed individually	£896–1,288	7×£171–205 = £1,197–1,435
7 devices, cheapest real bulk path	10-pack (over-buying): £647–932	6-pack + 1 single: £690–828
7 devices, hidden behind 1 router	£128–184	£171–205

Per theatre, that's roughly **5-7x cheaper** at ExCeL and **4-7x cheaper** at Olympia (the low end comparing against the cheapest bulk path, the high end against individual billing). Across all 14 GL-iNet routers in this design (12 theatres + 2 VMix nodes, each also with several devices behind it — see `docs/ip-address-map.md`), the gap runs into the thousands of pounds for the event, not a marginal saving.

Important — this isn't a loophole to just quietly exploit. Both venues' published technical policies explicitly prohibit exhibitors from bringing their own WiFi access points, mesh systems, or mobile hotspots onto the show floor without prior written authorization, and both reserve the right to detect and disconnect unauthorized hardware — this is standard, close-to-identical boilerplate at both ExCeL and Olympia, not an oversight. Using a GL-iNet router this way is a legitimate, common arrangement for professional AV vendors — but it means getting the venue's technical services team to authorize it ahead of time (registering the router, agreeing it's not itself broadcasting a public/guest SSID, etc.), not simply plugging one in on the day.

Sources: ExCeL Event Services Rate Card 2026 — IT, ExCeL Wireless Policy, London Book Fair 2025 eForce Internet Order Form (Olympia). Prices change; treat these as illustrative of the *shape* of the saving, confirm current figures with the actual venue before budgeting against them.

GLKVM-Cloud and remote administration

Already built into this design — see `docs/glkvmm-cloud.md` for the full detail. In short: a self-hosted instance of GL.iNet's own GLKVM-Cloud platform gives centralized, browser-based SSH terminal and web-admin proxy access to all 14 Slate AX routers at once, using GLKVM-Cloud's documented support for embedded OpenWrt devices — not its flagship KVM-hardware feature set, since there's no physical KVM unit in this design. Worth restating here as part of the broader case for this hardware family specifically: choosing GL-iNet doesn't just get 14 capable routers, it gets a fleet that's centrally manageable as one console instead of 14 separate admin sessions, for free, using tooling the vendor already publishes and this design already self-hosts.

Beyond this event: other places this pattern is useful

None of the following is used in this build — worth documenting anyway, since this is reusable hardware and the same GL-iNet-router-plus-Tailscale pattern generalizes well beyond a 12-theatre conference.

Remote administration of LED video processors (Novastar)

Novastar's LED processor/controller line is normally driven by a laptop physically present on the same local network as the hardware. A GL-iNet router with Tailscale turns that into remote access from anywhere — same principle as GLKVM-Cloud giving remote access to this design's own routers, applied to a different vendor's hardware.

Novastar MX30, via VMP. VMP (Vision Management Platform — not "Video Management Platform") is Novastar's control app for the COEX line the MX30 belongs to. Its own documentation describes adding a controller by **typing its IP address directly** ("Add Controller → enter the IP"), not by picking it off a broadcast-discovered list, and explicitly documents a router-mediated topology as one of its two supported connection methods ("PC and controller connected to the same LAN via a router... suitable for systems in complex environments"). Control traffic runs over TCP port 5200 (UDP 5201 for

the UDP variant), per Novastar's own Central Control Protocol documentation — a unicast, IP-addressed protocol, not a broadcast-dependent one, which is exactly the traffic pattern that crosses a Tailscale tunnel cleanly. VMP's manual never explicitly addresses VPN/WAN use (neither endorsing nor ruling it out) — the underlying protocol lining up cleanly with how Tailscale actually works is a real signal, not a confirmed guarantee. Novastar's own sanctioned path for genuine internet-native access is a separate product, **VNNOX** (a hosted cloud platform) — worth knowing about as the vendor's alternative, though it's a different management layer entirely, not "VMP over the internet." Sources: VMP User Manual, COEX Wiki — Device Connection and Setting, Central Control Protocol Instructions V1.5.0.

A note on "MCTRL880": that specific model doesn't exist in Novastar's current or past product catalog — the closest real sending cards are the **MCTRL660 / MCTRL660 Pro, MCTRL600, and MCTRL4K**. The rest of this section uses MCTRL4K, since that's also the model the Tailscale question below is about specifically; the same reasoning applies to the MCTRL660/600 family.

NovaLCT + MCTRL4K over Tailscale — checked specifically, genuinely uncertain, not a confirmed "yes." No official Novastar documentation, forum thread, or case study addresses this exact combination — worth being upfront about that rather than asserting it works. What's actually known, and why it points toward "probably, with a caveat":

- NovaLCT ships in different variants for different Novastar product families. For Novastar's **Multimedia Player** line (standalone async players, not sending cards), the official manual explicitly documents a cross-subnet workaround: *"If the terminal and NovaLCT are not on the same network segment but they can be pinged... select Specify IP, enter an IP address and click Search."* That's a confirmed, official, IP-based (non-broadcast) connection path — but for a different product family than the MCTRL sending cards.
- For the **Synchronous Control System** variant of NovaLCT — the one that actually talks to MCTRL4K/660/600-class sending cards — the manual only says NovaLCT "connects to the sending card automatically" once the hardware connection is normal, with no equivalent "Specify IP across a routed network" language found in any version checked. That phrasing suggests the initial device-discovery step may depend on local broadcast/ARP rather than a directable IP search — and Tailscale, being a routed L3 overlay rather than a bridged L2 network, **does not carry broadcast/multicast traffic between peers** (the same limitation this design already worked around for NDI — see `docs/streaming-flow.md`).
- Once a connection exists, though, MCTRL-family control traffic itself is unicast TCP to a known IP on port 5200 — confirmed by Novastar's own COEX protocol doc and independently corroborated by two separate community reverse-engineering projects (Bitfocus Companion's Novastar module, an independent Wireshark-based protocol writeup). That part should traverse a Tailscale tunnel without issue.

Practical read: the control *traffic* should work fine over Tailscale; the open question is purely whether NovaLCT's own device-*discovery* step for sending cards needs local broadcast to find the MCTRL4K in the first place. If NovaLCT's Synchronous Control System build has its own "add by IP" option (unconfirmed here — check the actual installed version), that sidesteps discovery entirely and is the path to try first. **The documented fallback that doesn't depend on any of this:** the MCTRL4K also exposes its own **web-based configuration UI**, reachable by pointing a browser at its IP — a plain HTTP-to-a-known-address connection with no broadcast dependency, and Novastar's own docs already describe reaching it this way (they just assume same-LAN, not a routed tunnel — no technical reason a browser would care about the difference). If NovaLCT itself doesn't cooperate over Tailscale, the web UI is the thing to actually try.

Remote WiFi access point for mixing desk control apps

A GL-iNet router can also just be a **portable, purpose-brought WiFi access point** — useful anywhere a venue's own WiFi is unreliable, absent, or not trusted for control traffic, letting an engineer run **Yamaha StageMix, Allen & Heath Qu-Pad, or Mixing Station** (a third-party app supporting many console brands) from an iPad without depending on house WiFi at all.

This is a genuinely different configuration mode from everything else in this document, worth being explicit about: everywhere else here, the GL-iNet router creates its **own routed, NAT'd subnet** with its own DHCP (that's the whole point of the per-theatre segmentation this doc opens with). For mixing-desk control, the safe default is the opposite — configure the router as a **plain bridged access point**, putting the iPad on the exact same L2 broadcast domain as the console, not a separate routed subnet behind it. That distinction is the crux of whether each app actually works:

App	Discovery mechanism	Manual IP fallback	Works across a routed subnet?
Yamaha StageMix (CL/QL-series consoles)	None — manual IP entry is the <i>only</i> method, and the app's own compatibility check assumes same-subnet addressing	Yes, but it's not actually a cross-subnet workaround here	Not supported per the official manual — no documented cross-subnet mode at all
Yamaha StageMix (TF-series and newer)	AUTO mode, mDNS-style, gated by iOS's Local Network permission	Yes — an explicit MANUAL mode	Yes, explicitly documented — connects across a different subnet, at the cost of losing live meter data
Allen & Heath Qu-Pad	Device list, almost certainly UDP broadcast per A&H's general control-protocol architecture	Not documented specifically for Qu-Pad (A&H's general platform docs say manual IP is the intended workaround when broadcast is blocked, but this isn't confirmed for Qu-Pad itself)	Probably, but under-documented — bench-test before relying on it live
Mixing Station	Varies by console brand — broadcast-based for Behringer X32/M32 (OSC/UDP) and Soundcraft (HiQNet, where the <i>mixer</i> initiates the connection to the app), inherited native behavior for others	Yes, universally — the app always supports typing a console's IP	Depends on the specific console's own protocol , not on Mixing Station itself

Bottom line: bridged AP mode is the one setup that works identically across all of these, no per-app or per-console workaround needed. A routed subnet is conditionally viable (TF-generation StageMix explicitly supports it; Mixing Station's manual-IP entry helps, but the underlying console protocol still might not) — treat it as something to test deliberately, not assume, if the router's other NAT/firewall features are wanted badly enough to be worth the added risk. For a one-off "get StageMix working from front of house" use case, bridged AP mode is the boring, reliable default. Sources: QL StageMix V8 User Guide, TF StageMix V4.5 User's Guide, Qu Series Reference Guide, Qu-Pad Help Guide, Mixing Station — Getting Started, Mixing Station — Soundcraft/HiQNet.

Mesh and peer-to-peer: fewer physical drops

Worth being precise about two different things both loosely called "mesh" here, since they operate at completely different layers and don't depend on each other:

- **Tailscale's own overlay mesh** — already how every router in this design reaches the mothership and each other, regardless of physical topology (see the Tailscale section above). It doesn't care whether the underlying WAN path is a dedicated wired drop, WiFi, cellular, or a hop through another GL-iNet unit — Tailscale just needs *some* IP reachability, wired or wireless, to establish its own tunnels on top.
- **GL-iNet's radio-layer repeater/WDS mode** — what this section is actually about: physically extending network reach between GL-iNet units over WiFi (or a wired Ethernet daisy-chain) instead of running a separate venue drop to every location. Worth naming accurately: GL-iNet markets this as repeater/extender (WDS) mode, not a self-healing mesh protocol in the eero/Orbi sense — one-directional dependency on whichever unit is upstream, not automatic multi-path failover between mesh nodes.

Running several adjacent theatres off a single network drop. One router lands the venue's wired uplink and acts as the "gateway" unit for a small cluster of adjacent theatres; the rest relay off it wirelessly (repeater mode) or via a wired Ethernet daisy-chain, while each theatre-side router still runs its own isolated subnet and DHCP scope exactly as described earlier in this document — the segmentation and Tailscale ACL isolation between theatres doesn't change, only how many theatres share one physical uplink. This directly trades against the "expensive per VLAN port" reality already central to docs/bandwidth-analysis.md : fewer wired drops needed from the venue, at a real cost.

That cost is worth stating plainly, because it partially undoes an argument made earlier in this document. The segmentation section above frames per-theatre subnets as independent fault domains — a DHCP problem or device fault in one theatre can't touch another. Meshing multiple theatres behind one shared uplink **reintroduces a shared fault point at the physical/WAN layer**: if the gateway unit's own uplink fails, every theatre relaying through it loses connectivity together, not just one. The subnets themselves stay isolated from each other (Tailscale ACLs don't change), but they stop being independently *reachable*. Worth treating as a deliberate, scoped trade-off — fewer drops needed, in exchange for correlated failure across whichever theatres share that one drop — not a strictly-better free option. Bandwidth is shared across the group too, same caveat as any repeater hop.

Going fully peer-to-peer, no wired backhaul at all. The extreme version of the same idea: no theatre gets a wired venue drop, and GL-iNet units relay to each other entirely over WiFi, hop by hop, until reaching whichever unit (or units) actually has a real WAN connection. Useful where running cable is genuinely impractical (temporary/pop-up spaces, venues that charge heavily per drop, awkward physical layouts) — but stacks the same shared-fault-point concern from above across every hop in the chain, plus WiFi's own range/interference/multi-hop-latency limits, which get worse the more hops are strung together. Reasonable for a small cluster of nearby rooms; not something to lean on for 12 theatres at once without real testing against the venue's actual RF environment first.

GL-iNet travel router comparison

Prices and specs below are current as of when this doc was written (mid-2026) — GL-iNet runs frequent promo pricing, and this isn't a live-updated table, so treat exact figures as indicative and re-check before procurement. GBP pricing was only reliably obtainable for a couple of models (GL-iNet's regional store pages don't consistently expose static GBP pricing to an automated check) — USD is the more solid figure throughout. The WireGuard column is GL.iNet's own figure for its built-in (kernel) WireGuard client; this design runs **Tailscale**, which uses its own userspace engine (`wireguard-go`) and is markedly slower on the same hardware — e.g. GL-MT1300 ~90 Mbps kernel WireGuard vs ~20 Mbps Tailscale (tailscale#10524), Beryl AX 300 vs ~150 Mbps each way (GL.iNet forum). No Slate AX Tailscale figure has been measured yet, see `docs/open-questions.md` #19.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
Slate AX (GL-AXT1800)	\$119.99 list	~550 Mbps (Tailscale ~150-250 Mbps combined — estimate, unbenchmarked)	2× GbE LAN, 1× GbE WAN, 1× USB 3.0	WiFi 6	USB dongle only	Yes (firmware 4.x Multi-WAN)	This build's choice since 2026-10-06 — see <code>docs/topology.md</code> . Qualcomm IPQ6000, 4× Cortex-A53 @ 1.2 GHz (64-bit); same 1 WAN + 2 LAN layout as the A-1300, so the wiring is unchanged.
Slate Plus (GL-A1300)	\$70–100 (promo vs. list — [<code>docs/bandwidth-analysis.md</code>] (bandwidth-analysis.md) uses ~\$100)	~170 Mbps (Tailscale est. only ~30-70 Mbps combined)	2× GbE LAN, 1× GbE WAN, 1× USB 3.0	WiFi 5	USB dongle only	Yes (firmware 4.x Multi-WAN)	Previous choice, rejected 2026-10-06: 32-bit IPQ4018 (4× Cortex-A7 ~717 MHz, 256 MB RAM), firmware stuck at 4.5.x with Tailscale 1.58; Go has no optimised ChaCha20-Poly1305 assembly there, so Tailscale can't carry even the default ingest scope. Smallest/cheapest of the group. Fallback only, running kernel WireGuard instead of Tailscale.
Beryl AX (GL-MT3000)	\$99	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1×	WiFi 6	USB dongle only (needs an add-on board for a real slot)	Yes (firmware 4.x Multi-WAN)	Only 1 LAN port — doesn't fit this build's "ATEM gets its own dedicated port" wiring without an added switch.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
			USB 3.0				
Flint 2 (GL-MT6000)	\$170	~900 Mbps	4× GbE + 2× 2.5G	WiFi 6	None (USB dongle/tethering)	Yes — failover + load-balancing, plus two Ethernet WANs	Highest throughput of the non-cellular models (Tailscale ~300-450 Mbps est.); much larger/heavier, poor fit for 12-per-theatre portability — the headroom option if the Slate AX benchmark comes in low.
Slate 7 (GL-BE3600)	\$150-170	~540 Mbps	1× 2.5G LAN, 1× 2.5G WAN, 1× USB 3.0	WiFi 7 (dual-band only — no 6GHz/320MHz, so limited real-world WiFi 7 benefit)	USB dongle only	Yes, per its own user guide	Newest of the group; compact. Only 1 LAN port — rejected for the same reason as the Beryl AX. (GL.iNet figures seen for its WireGuard rate range ~490-540 Mbps.)
Spitz AX (GL-X3000)	\$380	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	Built-in 5G/4G modem, dual Nano-SIM	Yes — Ethernet/repeater/cellular/tethering	No internal battery — needs external 12V power. The dedicated cellular-failover choice if built-in modem matters more than portability.
Puli AX (GL-XE3000)	\$410	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	Built-in 5G modem, dual Nano-SIM	Yes	Same cellular capability as Spitz AX, plus a built-in 6400 mAh battery — genuinely portable/untethered for short sessions, at a real price premium.

"Mesh" caveat: every model above advertises **repeater/extender (WDS) mode**, not a true self-healing mesh protocol between multiple units — worth being precise about this distinction rather than assuming parity with consumer mesh systems (eero, Orbi, etc.). See the mesh/peer-to-peer section above for what that means in practice for running several theatres off one drop.

For this build specifically, the Slate AX replaced the Slate Plus (A-1300) on 2026-10-06, after research found the A-1300's ~170 Mbps was GL.iNet's *kernel* WireGuard figure — Tailscale's userspace engine on its 32-bit CPU is estimated at only ~30-70 Mbps combined, short of even the default camera + program ingest scope (~89 Mbps combined, typical, per docs/bandwidth-analysis.md). The Slate AX keeps the same **2 LAN ports for the dedicated-ATEM-port wiring — still the harder constraint than raw throughput**, and why Beryl AX and Slate 7 remain traps regardless of their higher ceilings — on a 64-bit CPU with an estimated ~150-250 Mbps under Tailscale. That estimate is extrapolated from the Beryl AX measurement, not measured: benchmark one unit before buying 14. The scenario that actually bites is still a theatre's NDI fallback plus its ATEM ingest at the same time (~213 Mbps combined, typical) — over the bottom of that range, so the config-only mitigation (pause that theatre's ATEM ingest during the fallback) stays part of the design. Spitz AX/Puli AX are the models worth reaching for specifically *because* of their built-in cellular — i.e. exactly the multi-WAN-failover and WiFi-as-primary-link scenarios earlier in this document, where a wired drop isn't guaranteed or a genuine backup path matters more than shaving cost/size. **Flint 2 is the headroom option** if the Slate AX benchmark disappoints or a further stream is ever proposed: desktop-sized, but comfortable even with ingest running through an NDI fallback.

The Comet KVM range

GL-iNet's separate KVM-over-IP product line — not used in this build (see `docs/glkvm-cloud.md` for why: GLKVM-Cloud here is being used purely for router administration, no physical KVM hardware involved), but worth documenting on its own merits since it's the same self-hosted-cloud philosophy applied to a genuinely different problem: out-of-band access to a physical machine's console — BIOS, boot failures, a hung OS — independent of whether that machine's own network stack is even up. Same benefit already described for GLKVM-Cloud's router-admin use here: no dependency on GL.iNet's vendor cloud, works without a public IP, reachable over Tailscale just like everything else in this design.

Model	Price (USD)	Video	Ports	Cellular	Rack/multi-server	Notes
Comet (GL-RM1)	\$70–90	4K@30 claimed — one independent review reports the HDMI input actually caps around 2K@60; not independently confirmed either way here	1× GbE, HDMI in, USB-C/USB2 for KB/mouse emulation	No	Single-server only	The base model — noted (and ruled out of scope) in <code>glkvm-cloud.md</code> .
Comet Pro (GL-RM10)	\$180	4K@30, hardware H.264 encode, ~30-60ms latency	1× GbE, USB for KB/mouse, HDMI passthrough	No (WiFi 6 only)	Single-server	Adds a small onboard touchscreen and a USB "fingerbot" accessory for physical power-button control.
Comet X (GL-RM4PE)	\$270	4K@30	Quad HDMI/USB — up to 4 servers per unit, single PoE Ethernet port powers the whole unit	No	Yes — 10"/19" rack-mount	The one genuinely built for a server-room/rack deployment rather than one-off remote access — the closest fit if this line is ever used to manage a rack of machines instead of a single box.
Comet 5G (GL-RM10RC)	\$300	Up to 4K, hardware H.264 encode	1× GbE, HDMI in/out (loop), 2× USB-C, 1× USB 2.0	Built-in 5G RedCap + 4G LTE fallback	Single-server	The cellular-backed model — genuine out-of-band access even if the target machine's own network <i>and</i> the venue's primary internet are both down, since the KVM's own uplink doesn't depend on either. Directly mirrors the multi-WAN failover reasoning above, just applied to a KVM unit instead of a theatre router.
Comet Q (GL-RMQ1)	\$80–130	N/A — DisplayPort Alt-Mode passthrough	USB-C only	No	N/A	Different category, not a straight comparison: a mobile-device (phone/tablet/laptop) remote-control dongle, not a server HDMI KVM. Was still in crowdfunding as of research — confirm general-availability/shipping status before treating pricing as current retail.

If this line is ever adopted, Comet 5G is the standout for exactly the same reason Spitz AX/Puli AX stand out among the routers: a genuinely independent uplink, not sharing fate with whatever network the thing it's managing (or the venue) is on. Comet X is the pick if the target ever becomes "a rack of servers" rather than one box. Same purpose-built-over-general-purpose-workaround logic this repo already applies elsewhere — see `docs/birddog-play-rationale.md` for the same argument made about BirdDog Play vs. a laptop running VLC.

Summary

For this build specifically, GL-iNet Slate AX routers plus Tailscale earn their place on three independent grounds that all point the same direction: **segmentation** contains DHCP, multicast, and security faults to a single theatre instead of the whole event; **Tailscale** routes between those segmented subnets without depending on the venue's own physical network cooperating, and enforces cross-theatre isolation at the overlay regardless of how the underlying VLANs actually turn out; and **GLKVM-Cloud** gives centralized administration of all 14 units as one console, self-hosted, for free.

The features this build doesn't currently use — multi-WAN failover, WiFi-as-primary-link, built-in cellular, mesh/repeater backhaul — aren't hypothetical extras. They're exactly the reasons this is reusable hardware rather than single-event kit: the same units, the same Tailscale-mesh architecture, and the same self-hosted-cloud philosophy (GLKVM-Cloud for routers, the Comet line for out-of-band KVM) apply directly to remotely administering LED processors, running a WiFi AP

for mixing-desk control apps, or cutting the number of wired drops a venue needs to provide — all without re-deriving the architecture from scratch each time.